

What Humans Own, What LLMs Do, What Stable Cells Automate: The Labor Allocation Framework in CKS

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in “Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems” (Li, April 2026).

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 29 April 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Abstract

The Coordination Knowledge Substrate (CKS) pattern commits to three modes by which coordination work can be performed: humans performing it directly, LLMs performing it under human-authored orchestration rules, and stable cells largely automating it under those rules. The source paper develops this allocation in §2.3 and reapplies it in §6.3 and §7.4, but treats it as a presupposition rather than a commitment in its own right. This note formalizes the three-mode framework as the *labor allocation framework* of CKS, identifies the three architectural properties without which it collapses, and isolates the central derivation: that labor allocation is independent of authority allocation. The human-governed commitment fixes who holds authority over substrate content; the labor allocation framework names what work the architecture permits to be performed by which kind of writer. Substrate content written by an LLM under a rule is no less subject to human inspect/modify/override rights than substrate content written by a human directly. Naming the framework as a separate commitment is what makes downstream design decisions — most consequentially non-specialist governance, which §7.4 commits to as architectural — defensible as architectural properties rather than as deployment accidents.

1. Why labor allocation needs to be named as its own framework

The source paper develops the three-mode allocation in §2.3 ("CKS is not human-only work... What humans own is authority... Everything else is labor the architecture supports but does not demand"), reapplies it in §6.3 to defend Claim 4's cost model against the "humans writing substrate content is expensive" critique, and reapplies it again in §7.4 to distinguish non-specialist governance from non-specialist authorship. Across these applications, the framework is treated as a presupposition that other commitments depend on rather than as a commitment in its own right.

Naming it explicitly does work the source paper's distributed treatment cannot. When a deployment performs a kind of work in one mode rather than another, the architecture supports the choice without prescribing it; the framework is what makes "supports the choice" precise. When work moves between modes, the substrate content remains the same kind of content under the same authority structure; the framework is what guarantees the move does not silently change either. When a deployment is challenged on whether it is "really" CKS-coherent because of how it allocates work, the framework is what answers — the architecture's commitment is *that all three modes are available*, not *which mode*

any particular work belongs in.

2. The three modes, defined

The framework names three modes in which coordination work can be performed within a CKS deployment, distinguished by who or what performs the work, not by what kind of work is performed.

Mode 1 — Direct human labor. Humans perform the work themselves: reading and writing substrate content, exercising governance rights, authoring orchestration rules, designing schemas, building cells. Substrate content produced in Mode 1 is attributed to the human writer. Mode 1 includes the governance moments — rule authoring and direct override, in the sense formalized in the human-governed note — and any other direct authorship.

Mode 2 — LLM labor under orchestration rules. The LLM performs work as substrate mediator: reading what the cell authorizes, reasoning over that content, and writing substrate content as the cell's outputs, all under human-authored orchestration rules. Content produced in Mode 2 is attributed to the LLM-under-rule, with cell and rule references attached. The LLM operates here in the AI-as-substrate-mediator role §4.2 commits to: it does not act as autonomous agent, and does not produce terminal outputs that bypass the substrate.

Mode 3 — Stable-cell automation. Cells whose orchestration rules are stable enough that the cell executes routinely without human intervention beyond initial rule authorship. The cell still uses an LLM mediator for individual executions — Mode 3 is not non-LLM execution, it is LLM execution under rules whose stability has reduced per-execution human attention to near zero. Content produced in Mode 3 is attributed to the cell, with rule reference and LLM execution recorded. This is the mode §6.3 invokes when noting that "stable cells can be delegated to LLM origination with human inspection and override rights preserved rather than actively exercised."

The three modes differ in what they require of humans at the moment of work — Mode 1 at the work itself, Mode 2 at rule authoring and override, Mode 3 only at rule authoring and intervention — but the substrate content they produce is of the same architectural kind, governed by the same authority structure.

3. The three architectural properties that make all three modes coherent

The framework is not free; it depends on three commitments holding across all three modes. Without any of the three, it collapses into disconnected modes that cannot be compared, governed uniformly, or moved between.

Property A — Single substrate. All three modes write to the same substrate. There is not a "human substrate" and an "LLM substrate" and an "automated substrate"; there is one substrate, and the three modes differ in who or what wrote each piece of content. This is what the source paper's substrate-as-source-of-truth commitment guarantees.

Property B — Mode-independent authority. All three modes are subject to the same human-governed authority structure. Substrate content written by an LLM under a rule is no less

subject to human inspect/modify/override rights than substrate content written by a human directly; substrate content written by a stable cell is no less subject to those rights than either. The mode of labor does not change the authority structure. This is what the human-governed commitment guarantees, and what the human-governed note formalized — authority is exercised at the rule-authoring moment and the override moment, neither of which is mode-specific.

Property C — Attributable writer. All three modes produce substrate content that carries writer attribution. The path retraceability commitment requires that any piece of substrate content be traceable to the conditions under which it was written; the framework relies on this to make labor mode itself an inspectable property. A reader (human or LLM, in any later cell execution) can determine whether a given piece of content was written in Mode 1, Mode 2, or Mode 3, and can act on that information.

The three properties are jointly necessary: without A, the framework needs a cross-substrate reconciliation mechanism it does not have; without B, human-governed becomes mode-conditional in a way the source paper does not commit to; without C, labor mode becomes invisible to the substrate the framework organizes.

4. Labor allocation is not authority allocation

This is the central derivation in this note, and the property that makes the framework coherent.

Authority is not allocable. The human-governed commitment makes authority over substrate content non-negotiable: humans retain the inspect/modify/override rights at all times, regardless of which mode produced the content. There is no operation in CKS by which authority moves from humans to a non-human writer. What the architecture permits is *execution under rules*. An LLM that writes substrate content in Mode 2 is operating under orchestration rules a human authored; a stable cell that writes in Mode 3 is operating under rules a human authored at a moment that may now be in the past. In neither case has the human's authority transferred. The human can override the rule, override the content the rule produced, or both, at any time.

Labor is allocable. The same coordination work can in principle be performed in any of the three modes. Work performed in Mode 1 — a human drafting substrate content directly — may, once patterns visible in the human's drafts have been captured as orchestration rules, also be performed in Mode 2. Work in Mode 2 may, once rules have stabilized, also be performed in Mode 3. The substrate content the work produces is the same architectural kind of content regardless of which mode produced it; the difference is who performed the labor.

Why the distinction matters. Conflating labor with authority produces two opposite errors, both of which a single-statement framework can preempt directly.

The first error treats labor delegation as authority delegation. A deployment hesitates to use Mode 2 or Mode 3 because doing so would, on this misreading, mean "giving up control" of the resulting substrate content. The architecture's answer is no: control is preserved regardless of labor, because authority is mode-independent. A deployment that uses Mode 3 aggressively retains exactly the same inspect/modify/override rights as a deployment that performs the same work in Mode 1.

The second error treats authority retention as labor retention. A deployment refuses to move work out of Mode 1 because doing so would, on this misreading, mean "giving up oversight." The architecture's answer is again no: the loop the human-governed commitment requires is the rule-authoring loop and the override loop, not the per-execution loop. A deployment that performs work in Mode 3 has not abandoned oversight; it has restructured oversight to be exercised at the rule layer rather than at the per-execution layer — exactly what §3.3's portable phrase, *governance is an authority architecture, not a review workflow*, was introduced to make available.

5. What the framework does NOT prescribe

Stating the framework precisely requires being equally precise about what it does not commit to, since allowing these questions to be read into the framework would convert an architectural commitment into deployment prescription.

It does not prescribe which kinds of work belong in which mode. Whether a particular kind of work — drafting a regulatory specification, capturing a decision rationale, resolving a recurring conflict — belongs in Mode 1, Mode 2, or Mode 3 depends on the domain, the risk profile, the available LLM capability, and the deployment's preferences. The architecture supports all three modes for any kind of work the substrate can hold; the choice is the deployment's.

It does not prescribe a temporal trajectory between modes. The framework permits work to be performed in any mode and to move between modes in either direction. The architecture commits to no default progression and to no maturity model in which work "graduates" between modes. A deployment that performs all of its coordination work in Mode 1 is CKS-coherent; so is one that automates aggressively under Mode 3. Neither is more "advanced" than the other in any sense the architecture endorses.

It does not commit to automation as a goal. The framework permits stable-cell automation; it does not require it. Mode 3 is *available* in the architecture, not preferable to other modes. A deployment that never reaches Mode 3 has not failed to implement the framework.

It does not prescribe how rules should be authored, how cells should be built, or how stability should be measured. Those are implementation choices the architecture leaves to the deployment.

6. How the framework interacts with the other commitments

The framework does not stand alone; it depends on, and enables, several of the other CKS commitments.

It depends on the substrate–cell boundary. Cells are the locus of Mode 2 and Mode 3 labor: an LLM in Mode 2 executes within a cell defining what it can read, write, and under what rules; a stable cell in Mode 3 is the same kind of object with rules whose stability has reduced human intervention to near zero. Mode 1 humans operate directly on the substrate.

It depends on AI-as-substrate-mediator. §4.2's commitment that the LLM operates as substrate mediator is exactly the role Modes 2 and 3 require. In none of the three modes does the LLM act as

autonomous agent, terminal producer of outputs that bypass the substrate, or authority-bearer over substrate content.

It depends on linear-cost scaling. §6.3's application of the labor distinction is what the framework here generalizes: Modes 2 and 3 only release human labor at scale because governance cost is not size-proportional. If governance cost grew with substrate size, the framework's offer that humans can govern content they did not author would be hollow — the governing itself would consume the labor the framework released.

It enables non-specialist governance. This is the most consequential downstream property of the framework, and the one that makes the source paper's §7.4 commitment to non-specialist governance an architectural property rather than a deployment-specific outcome.

§7.4 carefully distinguishes non-specialist *governance* (the architectural commitment that a non-specialist can inspect, modify, and override the substrate and the orchestration rules in tools they already use) from non-specialist *authorship* and non-specialist *maintenance* (the labor of producing substrate content and of keeping it current). The first is an architectural commitment; the second and third are POC-demonstrated capabilities the architecture supports but does not require.

The labor allocation framework is what makes that distinction structurally coherent. Without it, non-specialist governance would collapse into non-specialist authorship: a non-specialist who could only govern content they themselves had written would be a governor whose reach is bounded by their authoring capacity. With the framework, the non-specialist authors orchestration rules at design time, governs the content those rules produce regardless of who or what wrote any particular piece of it, and exercises override at moments of their own choosing. The content may have been written in any of the three modes; in all three cases, the non-specialist's authority is the same authority, exercised the same way, over the same kind of content.

This is also what makes the bottom-up adoption property the source paper's POC instantiates an architectural property rather than a fortunate empirical observation. A team can begin entirely in Mode 1, author orchestration rules as patterns become visible, and let work under stable rules move to Mode 3 — all without changing the substrate's governance structure. At every point, the team's governance authority is exactly what it was at the start; what has changed is only the labor allocation.

7. Operational test

A system implements the CKS labor allocation framework if and only if all of the following are true at all times during the substrate's existence:

- 1 Substrate content carries writer attribution that distinguishes among Mode 1 (human), Mode 2 (LLM-under-rule), and Mode 3 (stable cell) writers, and the attribution is inspectable by humans with appropriate access.
- 2 All three writer modes are subject to the same human-governed authority structure: the inspect, modify, and override rights formalized in the human-governed note apply uniformly to substrate content regardless of which mode produced it.

- 3 Work can move between modes without changing the substrate content's structure or its authority status — only the writer attribution changes when the mode changes.
- 4 The deployment is free to allocate work across modes according to its own preferences; the architecture imposes no specific allocation, no required progression between modes, and no mode that must be present.
- 5 Orchestration rules under which Mode 2 and Mode 3 labor execute are themselves human-authored substrate content (per the human-governed note's operational test, condition 4), and remain inspectable and modifiable as such.

A system that fails any of (1)–(5) may still allocate labor across humans and LLMs in some sense, and may be a useful system, but it does not implement the CKS labor allocation framework.

Conclusion

The labor allocation framework names what the source paper's §2.3 disambiguation — *what humans own is authority... everything else is labor the architecture supports but does not demand* — commits to as a single architectural property, and what subsequent applications in §6.3 and §7.4 depend on. The framework is independent of the human-governed authority commitment: labor mode does not condition authority structure, and authority retention does not require labor retention. The two commitments together are what permit the bottom-up adoption trajectory the POC demonstrates and the non-specialist governance §7.4 commits to as architectural.

Implementations that read CKS as requiring direct human labor for all coordination work miss the framework's central point and produce systems that fail to scale, conflating authority retention with labor retention. Implementations that read CKS as authorizing autonomous LLM operation miss the distinction in the other direction and produce systems that violate the human-governed commitment. Naming the framework precisely is what makes both misreadings visible as misreadings. Subsequent work that adopts the CKS pattern, extends it, or composes it with adjacent patterns should use "the labor allocation framework" in the sense formalized here, alongside "human-governed" in the sense formalized in the prior note.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *What Humans Own, What LLMs Do, What Stable Cells Automate: The Labor Allocation Framework in CKS*. Derivation Note. 29 April 2026. ORCID: 0009-0004-8065-3235.